
On-Device Diffusion: A Diffusion-Aware Compiler and Runtime

Mahad Faruqi
Purdue EcoAI Lab
Advisor: Prof. Haoran You

1 Motivation

On-device diffusion inference must balance generation latency, peak memory, and output quality. Its repeated denoising computation creates opportunities to reuse intermediate results, specialize kernels, and manage tensor lifetimes across steps. These choices interact: caching saves computation but occupies memory, while lower precision changes both kernel availability and the space available for caches. SVDQuant demonstrates the importance of matching quantization to kernel implementation [15]. TeaCache and DiCache demonstrate adaptive approximate reuse across denoising steps [16, 6]. A useful deployment system must decide which supported combinations benefit a particular model, workload, and device.

Existing engines already address substantial parts of this problem. Stable-diffusion.cpp provides diffusion inference across hardware backends, and edge-dit.cpp includes memory-aware precision and placement [23, 24]. Compiler-based deployment also has precedents in TVM and MLC Web Stable Diffusion [8, 19]. The proposed contribution is therefore diffusion-specific planning within a compiler/runtime: representing denoising structure, exposing compatible optimization choices, and using measured costs and quality evidence to select and specialize an execution plan.

The central hypothesis is that jointly planning reuse regions, precision, and memory residency can reduce end-to-end latency under a fixed memory budget and declared quality requirement, compared with fixed configurations or independently selected policies. The study will test this hypothesis with a bounded optimization space, beginning with FLUX image generation and extending to Wan video generation. DreamLite adds a deliberately narrow phone workload: its 0.39B mobile U-Net, four-step distilled variant, and demonstrated iOS deployment show a different point in the model–system design space [9, 7]. It does not replace the execution-planning hypothesis but helps test which parts of the planner remain useful when the model was designed for a phone from the outset.

2 Target Models and Backends

The main workloads are FLUX.2 [klein] 4B text-to-image generation and Wan 2.1 T2V-1.3B text-to-video generation, including text encoding, denoising, and decoding [4, 25]. FLUX’s four-step configuration is a useful test of when caching overhead outweighs its benefit. Approximate-reuse experiments will include a longer denoising workload, using Wan and, where feasible, FLUX.2 [klein] 4B Base [5]. Video tests will start with short 480p clips and record frame count, frame rate, sampling schedule, and resolution.

DreamLite-mobile will be a bounded iPhone workload and deployment reference rather than a third full optimization study. DreamLite unifies text-to-image generation and text-guided editing in one pruned mobile U-Net by spatially concatenating target and source latents; progressive training and step distillation produce a four-step mobile variant [9]. Its iOS reference splits the pipeline across a 4-bit MLX text encoder and FP16 Core ML U-Net and VAE, coordinated by a Swift scheduler [7]. This supplies three useful tests for the proposed system: per-stage backend and precision choices must be first-class plan decisions, a four-step model may correctly select no approximate cache,

and model compression/distillation must remain distinct from execution-only improvement. Editing evaluation is optional and will not displace the core generation experiments.

Targets are the Jetson Orin Nano through CUDA, the M1 MacBook Pro through Metal, and an iPhone through an iOS-native path. An A100 also provides a CUDA development and reference platform. The provisional phone floor is an iPhone 16 Pro or Pro Max (8 GB RAM) or newer, matching DreamLite’s published deployment requirements. The exact model and OS version will be recorded [7]. Core ML can use the CPU, GPU, and Neural Engine, while MLX Swift provides a Metal-backed Apple-silicon path [2, 20]. Device records will include usable memory, enabled compute units, power mode where applicable, OS/runtime versions, and initial thermal state. Memory accounting will include the complete pipeline and shared physical memory: moving weights between CPU and GPU allocations does not by itself eliminate their footprint on Jetson [21].

TVM Relax and tensor-level lowering are the initial compiler foundation for supported CUDA and Metal paths [8, 14]. Model adapters will expose the denoising loop and its dependencies explicitly, importing a denoiser graph alone does not reveal the surrounding sampling process. The iPhone path begins from DreamLite’s Core ML/MLX exports and Swift runtime, then exposes their per-stage choices through the same plan schema. This tests planner portability without claiming that one compiler already lowers every stage to every mobile accelerator. Quantized representations derived from a fixed checkpoint will be tracked separately from distilled or replacement checkpoints. The planner will never silently replace the requested model: an infeasible request produces a reported limitation or an explicitly selected alternative workload.

3 Pipeline Architecture

On-Device Diffusion

Discover legal choices → choose for this device → prepare once → execute and learn locally

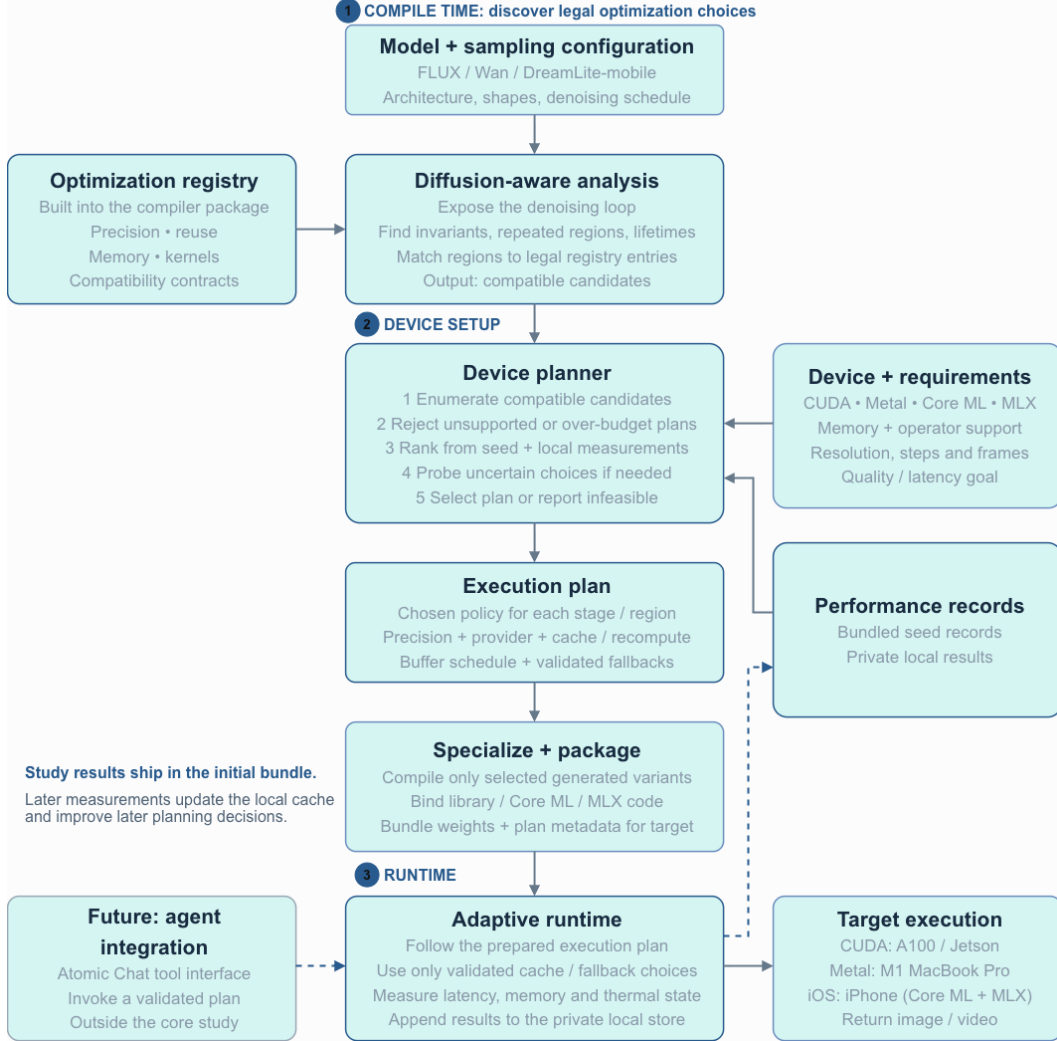


Figure 1: Compile time discovers legal optimization choices. Device setup rejects infeasible combinations, selects an execution plan from bundled study results and a local measurement cache, and specializes a package. Runtime follows that plan and updates the cache for later requests.

Compile-time analysis. Model adapters import stage graphs into Relax and describe the timestep loop, conditioning dependencies, and sampling configuration. Analysis identifies repeated regions, shapes, tensor lifetimes, and step-independent computations. Invariance requires all relevant inputs to remain unchanged. Text-token features that interact with changing image features are not automatically invariant. The analysis supplies candidate boundaries and storage estimates. Slowly changing features are candidates for approximation but not compiler-proven invariants.

Optimization registry. Each implementation declares its supported architectures, devices, dtypes and layouts, candidate regions, required state, memory and runtime costs, transformation or kernel hooks, and compatibility constraints. Exact transformations and approximate policies have distinct contracts. Approximate entries additionally record calibration requirements, applicable quality evidence, and runtime guards. The initial families are precision, cross-step reuse, memory management,

and kernel selection. TeaCache and DiCache are interchangeable reuse implementations where supported [16, 6]. New methods require implementation and validation. Methods that change execution semantics may also require extending the interface. The design follows the separation of compiler passes, dispatch, and runtime mechanisms illustrated by MLC-LLM [17], without assuming that its LLM pipeline already supports diffusion.

Deployment-time planner. The planner turns compiler facts into a device-specific choice through four explicit steps. First, it enumerates combinations exposed by the analysis and registry, such as per-stage precision, execution provider, residency, and supported reuse regions. Second, it rejects combinations whose operators are unsupported, whose policies conflict, or whose estimated peak memory exceeds the budget. Third, it ranks the survivors using seed measurements bundled from this study and measurements in the local cache, then benchmarks a small number of uncertain candidates if needed. Finally, it selects one feasible plan or reports that the requested model and workload cannot run under the declared constraints. A lightweight regressor trained on accumulated measurements will be compared with rules and measured lookup when sufficient data are available. Full generations on separate calibration prompts establish quality eligibility and the planner does not need to execute every policy for every new prompt.

For compatible plans $\mathcal{P}$, the objective is

$$p^* = \arg \min_{p \in \mathcal{P}} \widehat{L}(p) \quad \text{subject to} \quad \widehat{M}(p) + M_{\text{reserve}} \leq B, \quad p \in \mathcal{P}_Q,$$

where B is the available memory budget and $\mathcal{P}_Q$ contains plans meeting the declared quality criteria on calibration data. Quality eligibility is empirical and workload-specific, not a guarantee for every prompt. Peak-memory estimates include weights, activations, workspaces, caches, and loading transitions. Combinations are measured together because their costs and errors need not be additive. No-reuse execution remains a candidate, and caching is rejected when its memory or decision overhead exceeds its benefit.

Persistent profiling and history. History uses two local sources. An initial seed database, produced from this study’s controlled experiments, is versioned and bundled with the model or application release. A writable local cache records plan identifiers, stage/region timings, peak memory, and failures from later runs. Both are keyed by checkpoint and graph fingerprints, workload settings, device, compiler/runtime version, and policy version. The runtime updates the cache after a request, and the next planning decision can use those measurements.

This resembles existing compiler caches but operates at a different level. TVM MetaSchedule persists measured tensor schedules in a local database and can reuse structurally matching records [22, 1]. MLC packaging similarly uses a local JIT cache to avoid recompiling the same inputs [18]. The proposed stores instead describe whole diffusion plans and keep empirical quality eligibility separate from performance records. A changed model, device, runtime, or policy invalidates the relevant records and triggers conservative defaults or bounded re-profiling.

Specialization and adaptive runtime. The execution plan is the explicit contract selected for one model, workload class, and target. It names the precision and provider for each stage, legal reuse regions and thresholds, tensor residency and buffer lifetimes, and prevalidated fallback variants. Specialization realizes that contract: it compiles only the selected generated variants, binds compatible library/Core ML/MLX implementations, and packages them with weights and plan metadata for the target.

The adaptive runtime follows this prepared plan rather than inventing a new one during generation. Previous runs influence it indirectly because their cached measurements helped the planner choose the plan. During a request, the runtime may make only the bounded decisions authorized by that plan, such as refreshing a cache, evicting optional state, or switching to a prepared state-compatible fallback. It monitors resources and policy signals while executing encoding, repeated denoising, and decoding. Refreshing a cache cannot undo error already introduced into the latent trajectory, and unvalidated quality changes are not used to force a request to fit. Afterward, performance measurements update the local cache for later planning.

4 Implementation Approach and Optimization Scope

The prototype will reuse compiler infrastructure, quantizers, kernels, and published reuse policies. Four registry families organize the candidate space:

- **Precision:** supported full/half-precision and low-bit representations, with per-component settings and compatible kernels. SVDQuant is a candidate where its implementation can be integrated [15]. Low-bit storage alone does not imply faster execution.
- **Reuse:** exact reuse of proven step-independent values, no reuse, and one approximate policy initially. A second policy tests whether the registry and planner remain independent of a particular method.
- **Memory:** buffer reuse, stage residency, selective recomputation, and tiled decoding where supported [13]. Choices account for transfers and the target’s physical memory organization.
- **Kernels:** existing generated or library implementations, including fused attention and compatible low-bit operations. Kernel support constrains precision and region boundaries.

The minimum research prototype will jointly choose a small set of reuse regions and memory policies across supported precision configurations. Other families provide compatible choices rather than independent optimization projects. Region selection will compare saved computation with persistent storage, probe overhead, and observed quality loss. Only boundaries supported by the chosen reuse implementation will be exposed. One-device results will precede cross-device and video transfer tests.

5 Research Questions and Expected Contribution

1. **RQ1: Which diffusion optimization choices transfer across devices and workloads?** Measure the compute, memory, and quality tradeoffs of precision, reuse, residency, and stage placement across CUDA, macOS Metal, and iOS Core ML/MLX, including whether caching remains beneficial for few-step image generation and longer video runs.
2. **RQ2: Can structure, hardware information, and persistent history select effective plans at low cost?** Compare rule-based selection, measured lookup, and a learned cost model using equal profiling budgets. Test held-out workloads and report prediction error, search cost, constraint violations, and latency relative to the best evaluated feasible plan.
3. **RQ3: Does joint planning with bounded runtime adaptation outperform fixed or independently tuned policies?** Isolate region selection, memory planning, and runtime guards under matched checkpoints and quality requirements. Identify where saved computation outweighs cache storage, probe costs, and runtime overhead.

Expected outputs are a diffusion-aware analysis and policy interface, a resource planner with persistent profiling, and an adaptive execution prototype with controlled ablations. The thesis will characterize benefits and failure cases, including cases where simpler policies or no reuse remain preferable.

6 Evaluation Plan

Baselines and controls. Compare supported stable-diffusion.cpp and edge-dit.cpp configurations, including their automatic fitting options; video baselines include the Wan reference implementation and compatible FastVideo paths [23–25, 11]. On iPhone, use the published DreamLite-mobile Core ML/MLX configuration as the end-to-end baseline, then compare planner-selected supported variants within that stack [7]. When feasible, run the same DreamLite-mobile artifacts on M1 and iPhone to isolate device transfer. DreamLite and FLUX results remain separate model-level studies. Within each stack, compare a conservative no-reuse plan, a tuned fixed configuration, independently tuned policies, and the joint planner. An offline bounded sweep defines the best evaluated feasible plan, not an unattainable global oracle. Keep prompts, initial noise where supported, resolution, schedule, guidance, and video settings matched. Report changed checkpoints and unsupported combinations separately.

Quality and selection discipline. Use separate calibration and held-out prompts/seeds. Before planner tuning, freeze compact and fidelity-focused quality criteria: maximum allowed degradation from each model’s reference on GenEval for image alignment [10], selected VBench dimensions for video consistency and quality [12], and structured blinded visual review. Record metric thresholds, prompt counts, and review criteria with each experiment. Paired perceptual differences can diagnose approximation error but do not replace alignment or visual-quality assessment. Evaluate combined policies together, report uncertainty and failure rates, and present latency–memory–quality tradeoffs where matched-quality comparisons are unavailable.

Latency, memory, and overhead. Measure full generation and individual stages, separating compilation/export, calibration, planning, cold loading, and warm inference. Include transfers, synchronization, runtime probes, and cache management. Report repeated timings with median, tail latency, and uncertainty; peak device allocations and total physical memory are both relevant on shared-memory targets. For iPhone, record initial and final thermal state, sustained-run latency, package size, and energy impact where the available tooling permits. Record out-of-memory failures, fallback frequency, and resource headroom. Measure how many repeated requests amortize preparation and profiling costs.

Planner and runtime ablations. Remove history, diffusion-specific analysis, joint policy selection, and runtime adaptation in turn while holding candidate choices and profiling budgets fixed where applicable. For history, compare cold-start and warm-start planning and validate model/device/version matching. Freeze the database and predictor before held-out tests; separately evaluate sequential cache updates using only measurements available before each decision. Controlled memory-pressure tests assess guard overhead and fallback behavior. Existing adaptive caching methods remain active in their corresponding baselines so their benefits are not incorrectly attributed to the new planner.

7 Milestones and Feasibility

Weekly implementation schedule. The schedule remains September 28–December 11, 2026. Difficulty ranges from 1 (easier) to 5 (more uncertain). Continued writing and measurement logging.

Week / dates	Goal	Deliverable	Level
1: Sep 28–Oct 2	Establish A100 and Jetson image baselines; review DreamLite and define the bounded planning hypothesis.	Baselines and scope	3/5
2: Oct 5–9	Add M1 and A100 Wan baselines; verify iPhone/DreamLite export and device access; specify registry and history records.	Evaluation protocol	3/5
3: Oct 12–16	Compile a denoising region; expose loop dependencies and candidate boundaries; verify target feasibility.	Analysis prototype	4/5
4: Oct 19–23	Connect stages and one reuse policy; add per-stage provider metadata and iOS measurement capture.	Instrumented pipeline	5/5
5: Oct 26–30	Implement bounded joint search using rules and measured costs; prepare compatible execution variants.	Planner prototype	5/5
6: Nov 2–6	Add runtime guards and a second policy; compare learned ranking with lookup if data permit.	Adaptive prototype	4/5
7: Nov 9–13	Evaluate transfer across CUDA, macOS Metal, and iOS Core ML/MLX; isolate integration overhead.	Transfer results	4/5
8: Nov 16–20	Run planner, history, and runtime ablations; test resource pressure, iPhone sustained runs, and preparation amortization.	Core comparisons	4/5

Week / dates	Goal	Deliverable	Level
9: Nov 23–27	Resolve any pending issues and freeze configurations, history, and evaluation scripts.	Frozen experiments	2/5
10: Nov 30–Dec 4	Run held-out quality tests and repeated timings; complete matched-quality comparisons.	Final results	4/5
11: Dec 7–11	Reproduce key results and compile a report.	Thesis (draft) and artifacts	3/5

Fallbacks and scope control. If FLUX does not fit Jetson or iPhone, first test supported quantization, sequential stage loading, and tiled decoding. A lower resolution or smaller/replacement checkpoint becomes a separately labeled workload. DreamLite-mobile supplies the phone-feasible baseline won’t be treated as an optimized FLUX result. Unsupported compiler operators may use decompositions or existing-runtime stages, with integration costs measured. If direct iOS compiler lowering remains blocked, retain the Core ML/MLX artifacts behind the shared plan interface and we will narrow the cross-backend compilation claim. Video and cross-device coverage will be limited to feasible configurations. A learned predictor is an evaluated alternative, not a dependency for the rule/measurement-based planner.

Future work: agent integration. A local agent could invoke validated plans through a structured tool interface, using Atomic Chat’s stable-diffusion.cpp integration as an application reference [3]. Joint language-model/diffusion residency and agent-selected quality profiles remain future work, outside the core implementation and evaluation schedule.

References

- [1] Apache TVM contributors. MetaSchedule: Search-based auto-tuning. Apache TVM documentation. URL https://tvm.apache.org/docs/deep_dive/tensor_ir/tutorials/meta_schedule.html. Accessed September 23, 2026.
- [2] Apple. Core ML developer documentation. Online documentation. URL <https://developer.apple.com/documentation/coreml>. Accessed September 23, 2026.
- [3] AtomicBot-ai contributors. Atomic Chat: Image generation with stable-diffusion.cpp. GitHub pull request 300, 2026. URL <https://github.com/AtomicBot-ai/Atomic-Chat/pull/300>. Merged September 21, 2026; accessed September 23, 2026.
- [4] Black Forest Labs. FLUX.2 [klein] 4B: Model card. Hugging Face model documentation, 2026. URL <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>. Accessed September 21, 2026.
- [5] Black Forest Labs. FLUX.2 [klein] 4B Base: Model card. Hugging Face model documentation, 2026. URL <https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B>. Accessed September 21, 2026.
- [6] J. Bu, P. Ling, Y. Zhou, Y. Wang, Y. Zang, T. Wu, D. Lin, and J. Wang. DiCache: Let diffusion model determine its own cache, 2025. URL <https://arxiv.org/abs/2508.17356>.
- [7] ByteVisionLab. DreamLite Mobile Deployment Guide: Swift, Core ML, and MLX. GitHub repository documentation, 2026. URL <https://github.com/ByteVisionLab/DreamLite/blob/main/deploy/README.md>. Accessed September 23, 2026.
- [8] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594. USENIX Association, 2018. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.

- [9] K. Feng, Y. Wei, B. Chen, Y. Pan, H. Ye, S. Liu, C. Yan, and Y. Gao. DreamLite: A lightweight on-device unified model for image generation and editing, 2026. URL <https://arxiv.org/abs/2603.28713>.
- [10] D. Ghosh, H. Hajishirzi, and L. Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment, 2023. URL <https://arxiv.org/abs/2310.11513>.
- [11] Hao AI Lab and FastVideo contributors. FastVideo: A unified inference and post-training framework for accelerated video generation. GitHub repository. URL <https://github.com/hao-ai-lab/FastVideo>. Accessed September 23, 2026.
- [12] Z. Huang, Y. He, J. Yu, F. Zhang, C. Si, Y. Jiang, Y. Zhang, T. Wu, Q. Jin, N. Chanpaisit, Y. Wang, X. Chen, L. Wang, D. Lin, Y. Qiao, and Z. Liu. VBench: Comprehensive benchmark suite for video generative models, 2023. URL <https://arxiv.org/abs/2311.17982>.
- [13] Hugging Face. Diffusers documentation: Reduce memory usage. Online documentation. URL <https://huggingface.co/docs/diffusers/optimization/memory>. Accessed September 21, 2026.
- [14] R. Lai, J. Shao, S. Feng, S. S. Lyubomirsky, B. Hou, W. Lin, Z. Ye, H. Jin, Y. Jin, J. Liu, L. Jin, Y. Cai, Z. Jiang, Y. Wu, S. Park, P. Srivastava, J. G. Roesch, T. C. Mowry, and T. Chen. Relax: Composable abstractions for end-to-end dynamic machine learning, 2023. URL <https://arxiv.org/abs/2311.02103>. Revised February 2025.
- [15] M. Li, Y. Lin, Z. Zhang, T. Cai, X. Li, J. Guo, E. Xie, C. Meng, J.-Y. Zhu, and S. Han. SVDQuant: Absorbing outliers by low-rank components for 4-bit diffusion models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.05007>.
- [16] F. Liu, S. Zhang, X. Wang, Y. Wei, H. Qiu, Y. Zhao, Y. Zhang, Q. Ye, and F. Wan. Timestep embedding tells: It’s time to cache for video diffusion model, 2024. URL <https://arxiv.org/abs/2411.19108>.
- [17] MLC AI contributors. MLC-LLM: Compiler pass pipeline. GitHub source code, . URL https://github.com/mlc-ai/mlc-llm/blob/main/python/mlc_llm/compiler_pass/pipeline.py. Accessed September 23, 2026.
- [18] MLC AI contributors. MLC-LLM: Package libraries and weights. Online documentation, . URL https://llm.mlc.ai/docs/compilation/package_libraries_and_weights.html. Accessed September 24, 2026.
- [19] MLC AI contributors. Web Stable Diffusion: Bringing stable diffusion models to web browsers. GitHub repository, . URL <https://github.com/mlc-ai/web-stable-diffusion>. Accessed September 21, 2026.
- [20] MLX contributors. MLX Swift: A swift api for MLX on apple silicon. GitHub repository. URL <https://github.com/ml-explore/mlx-swift>. Accessed September 23, 2026.
- [21] NVIDIA. CUDA for Tegra: Memory management and porting considerations. Online documentation. URL <https://docs.nvidia.com/cuda/cuda-for-tegra-appnote/index.html>. Accessed September 21, 2026.
- [22] J. Shao, X. Zhou, S. Feng, B. Hou, R. Lai, H. Jin, W. Lin, M. Masuda, C. H. Yu, and T. Chen. Tensor program optimization with probabilistic programs. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2205.13603>.
- [23] stable-diffusion.cpp contributors. stable-diffusion.cpp: Diffusion model inference in C/C++. GitHub repository. URL <https://github.com/leejet/stable-diffusion.cpp>. Accessed September 21, 2026.
- [24] THU-MIG. edge-dit.cpp: A native C/C++ inference engine for diffusion transformers. GitHub repository. URL <https://github.com/THU-MIG/edge-dit.cpp>. Accessed September 21, 2026.

- [25] Wan Team. Wan2.1-T2V-1.3B: Model card and reference implementation. Hugging Face model documentation, 2025. URL <https://huggingface.co/Wan-AI/Wan2.1-T2V-1.3B>. Accessed September 23, 2026.